UNIVERSIDAD RAFAEL LANDÍVAR

INFORME TÉCNICO

PROYECTO 2 - MANEJO E IMPLEMENTACIÓN DE ARCHIVOS

AGENDA DE CONTACTOS

Nombre: Santiago Jefté Batz Rodríguez
Carné: 1548725

Quetzaltenango, 9 de noviembre de 2026

Descripción y organización

Agenda de Contactos es una aplicación de escritorio para agregar, buscar y modificar contactos. Su objetivo es trabajar con registros de tamaño fijo y encontrar un contacto por ID sin recorrer todo el archivo de datos.

Funciones principales

Sección	¿Qué permite hacer?
Contactos	Registrar, buscar por ID, mostrar la lista y editar información.
Exportar	Generar contactos.json y un reporte PDF basado en ese JSON.
Herramientas	Verificar SHA-256, crear respaldos ZIP y consultar propiedades del archivo.
Ayuda	Mostrar instrucciones básicas e información sobre el sistema.

Organización interna



Ruta	Uso
app/core/	Guarda y busca contactos, valida los registros.
app/services/	Exporta archivos y realiza comprobaciones y respaldos.
app/ui/	Contiene las ventanas y los botones de la aplicación.
datos/	Conserva contactos.dat, indice.json y contactos.sha256.
exportaciones/ · respaldos/	Guarda el JSON, el reporte PDF y las copias ZIP.

¿Cómo se guardan los contactos?

Todos los registros tienen el mismo tamaño: 168 bytes. Cada contacto tiene cinco campos: ID, nombre, teléfono, correo y ciudad, el programa convierte los datos a UTF-8 y rellena con espacios los campos que no usan todo su espacio.



COMPROBACIÓN SENCILLA

```
CAMPOS = {
    "id": 7,
    "nombre": 48,
    "telefono": 16,
    "correo": 64,
    "ciudad": 32
}

TAMANIO_REGISTRO = sum(CAMPOS.values()) + 1
```

Cada campo tiene un espacio establecido en bytes. Al sumar sus tamaños y agregar el salto de línea, cada contacto ocupa exactamente 168 bytes.

Campo	Espacio	Posición dentro del registro
ID	7 bytes	0–6
Nombre	48 bytes	7–54
Teléfono	16 bytes	55–70
Correo	64 bytes	71–134
Ciudad	32 bytes	135–166
Salto de línea	1 byte	167
TOTAL	168 bytes	0–167

¿Por qué un tamaño fijo?

Si todos los registros ocupan 168 bytes, se puede calcular dónde empieza cualquiera, or ejemplo, el registro 2 (contando desde cero) comienza en el byte 336.

COMPROBACIÓN SENCILLA

```
TAMANIO_REGISTRO = sum(CAMPOS.values()) + 1  # 168 bytes
```

El sistema también revisa que los campos no estén vacíos, que el ID sea válido y que la información no supere el espacio asignado. El tamaño se mide en bytes, no solamente en caracteres.

COMPROBACIÓN SENCILLA

```
datos = texto.encode("utf-8")

if len(datos) > tamanio:
    raise ValueError(f"El campo supera los {tamanio} bytes.")

return datos.ljust(tamanio, b" ")
```

Antes de guardar un dato, se convierte a UTF-8 y se comprueba que no supere el tamaño permitido. Si ocupa menos espacio, se completa con espacios en blanco.

Índice y acceso aleatorio

A. Agregar un contacto - Archivo: app/core/agenda.py

```
with open(ARCHIVO, "ab") as archivo:
    posicion = archivo.tell()
    archivo.write(registro)
    archivo.flush()
    os.fsync(archivo.fileno())

indice[str(identificador)] = posicion // TAMANIO_REGISTRO
```

Al agregar un contacto, se escribe al final del archivo. Con tell() se obtiene la posición donde comienza y se utiliza para guardar su número de registro en el índice JSON.

B. Buscar un contacto mediante el índice

```
posicion = indice[clave] * TAMANIO_REGISTRO

with open(ARCHIVO, "rb") as archivo:
    archivo.seek(posicion)

    posicion_actual = archivo.tell()
    registro = archivo.read(TAMANIO_REGISTRO)
```

El índice permite conocer el número de registro del contacto. Con ese dato se calcula su posición, seek() lleva el cursor directamente hasta allí y tell() indica la posición alcanzada.

C. Modificar un contacto

```
contacto.update(nuevos_datos)
registro = empaquetar_contacto(contacto)

with open(ARCHIVO, "r+b") as archivo:
    archivo.seek(posicion)
    archivo.write(registro)
    archivo.flush()
    os.fsync(archivo.fileno())

actualizar_hash()
```

Para modificar un contacto, primero se prepara el registro con los nuevos datos. Después se sobrescribe únicamente en su posición correspondiente y se actualiza el SHA-256.

Índice y acceso aleatorio

El archivo indice.json relaciona cada ID con un número de registro, al buscar un contacto, se consulta ese índice, se calcula el byte inicial y se lee únicamente el registro correspondiente.



seek() lleva el cursor a la posición calculada. tell() indica dónde está el cursor, después se leen los 168 bytes del contacto.

Modificación en el mismo lugar

Al editar, el programa sobrescribe ese registro. No vuelve a escribir todos los contactos y, al terminar, actualiza el SHA-256.

Búsqueda secuencial frente a búsqueda con índice

Secuencial	Con índice
Revisa los registros del .dat hasta encontrar el ID.	Carga el índice JSON y salta directamente al registro del .dat.

El índice se carga desde JSON en cada búsqueda, por lo que también tiene un costo, su ventaja aquí es evitar el recorrido completo del archivo de contactos.

Exportación de datos

Archivo: app/services/exportacion.py

A. Exportación a JSON

```
def exportar_json():
    contactos = obtener_contactos()
    CARPETA_EXPORTACION.mkdir(exist_ok=True)

    with open(ARCHIVO_JSON, "w", encoding="utf-8") as archivo:
        json.dump(contactos, archivo, indent=4, ensure_ascii=False)

    return ARCHIVO_JSON
```

Se recuperan los contactos almacenados y se guardan en un archivo JSON. Este formato permite consultar la información de manera más sencilla.

B. Generación del reporte PDF

```
with open(ARCHIVO_JSON, "r", encoding="utf-8") as archivo:
    contactos = json.load(archivo)

if not isinstance(contactos, list):
    raise ValueError("El archivo JSON no tiene el formato esperado.")
```

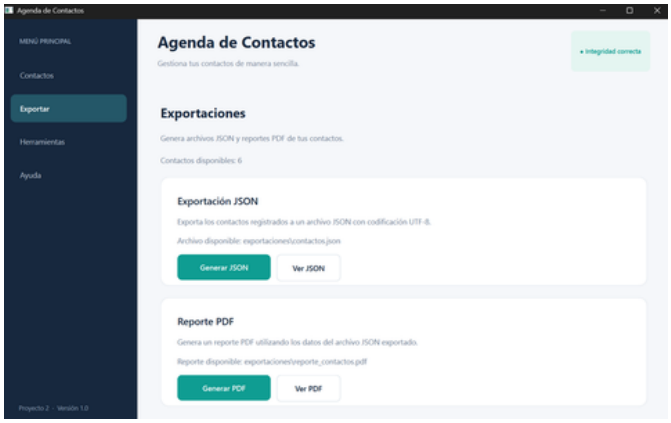
Antes de generar el reporte PDF, el programa lee la información del JSON y comprueba que tenga el formato esperado. Después utiliza esos datos para elaborar la tabla de contactos.

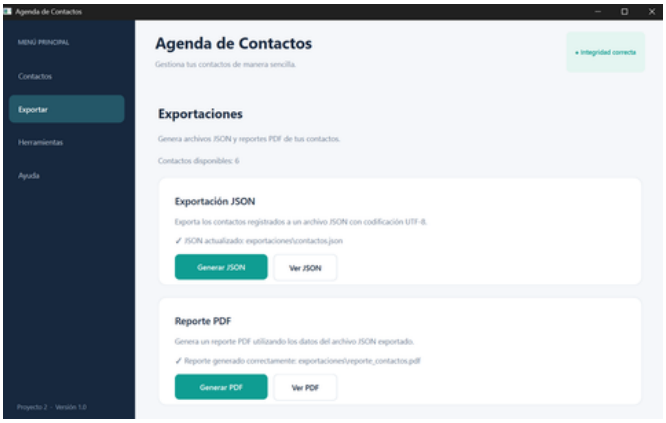
Exportación de datos

La sección Exportar permite generar contactos.json con todos los contactos, el reporte_contactos.pdf se crea después utilizando ese JSON, incluye una tabla y la fecha de generación.



Archivo	Qué contiene
exportaciones/contactos.json	Lista de contactos en un formato fácil de leer y compartir
exportaciones/reporte_contactos.pdf	Tabla de contactos y fecha de generación, usando los datos del JSON.







AGENDA DE CONTACTOS

Reporte general de contactos

Fecha de generacion: 08/11/2026 20:27

Total de contactos: 6

ID	Nombre	Telefono	Correo electronico	Ciudad
101	José Alejandro López	45678912	jose.lopez@correo.url.edu.gt	Retalhuleu
205	María Fernanda Pérez	59874123	maria.perez@correo.com	Coatepeque
310	Carlos Méndez	47895612	carlos.mendez@correo.com	Retalhuleu
405	Andrea Morales	45678901	andrea@correo.com	Quetzaltenango
333	Ray Ostia	31789978	ray@gmail.com	Ciudad de Guatemala
700	Jeremy Tobias	78849136	jeremy@correo.com	Escuintla

Integridad del archivo

Archivo: app/services/herramientas.py

```
sha256 = hashlib.sha256()

with open(ARCHIVO, "rb") as archivo:
   while True:
      bloque = archivo.read(4096)

      if not bloque:
         break

      sha256.update(bloque)

return sha256.hexdigest()
```

Se lee el archivo de contactos por bloques y se calcula su valor SHA-256. Este resultado se utiliza para comprobar si el contenido ha cambiado.

comparación

```
  hash_actual = calcular_hash()

  if hash_actual == hash_guardado:
     return True, "La integridad del archivo es correcta."

  return False, "Se detectaron modificaciones en contactos.dat."
```

El programa compara el hash calculado con el que se encuentra guardado. Si los valores son diferentes, informa que el archivo fue modificado.

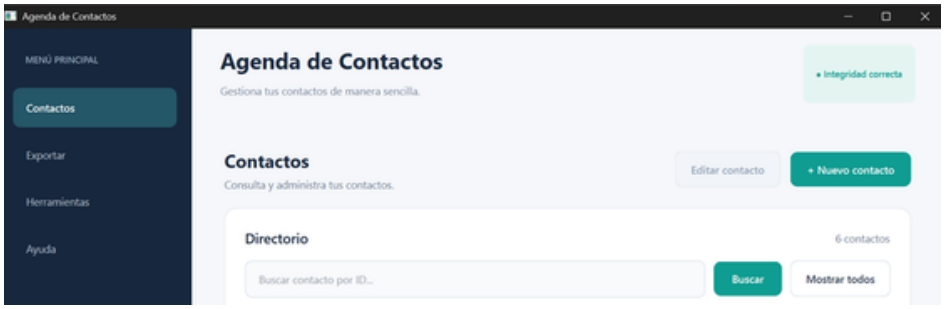
Integridad del archivo

Después de agregar o modificar un contacto, el sistema calcula el SHA-256 de contactos.dat y lo guarda en contactos.sha256. Cuando se abre la aplicación, vuelve a calcularlo y compara los dos valores.

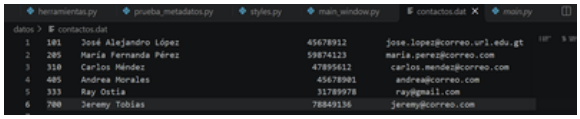
Si coinciden	Si no coinciden
Se muestra el estado Integridad correcta	Se avisa que el archivo pudo cambiar fuera de la aplicación
La agenda permite continuar trabajando	Se impide agregar o modificar contactos hasta resolver el problema

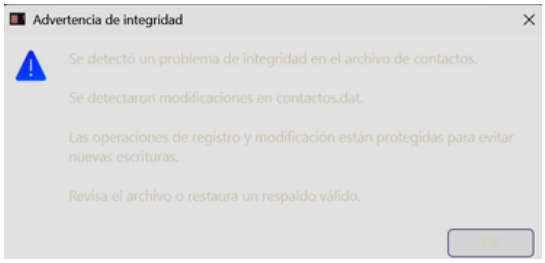
La prueba de alteración se hizo cambiando manualmente el archivo, el sistema detectó la diferencia y después fue posible recuperar una copia válida desde un respaldo.

Integridad correcta



Archivo alterado







Nota: SHA-256 permite detectar cambios, no dice quién modificó el archivo.

Respaldo ZIP y metadatos

Archivo: app/services/herramientas.py

A. Generación del respaldo ZIP

```
with zipfile.ZipFile(ruta_zip, "w", zipfile.ZIP_DEFLATED) as zip_archivo:
    zip_archivo.write(ARCHIVO, arcname=ARCHIVO.name)
    zip_archivo.write(INDICE, arcname=INDICE.name)

tamanio_original = ARCHIVO.stat().st_size + INDICE.stat().st_size
tamanio_comprimido = ruta_zip.stat().st_size
```

El programa crea un archivo ZIP que contiene los contactos y su índice JSON. Después obtiene el tamaño original y el tamaño comprimido para compararlos.

B. Consulta de metadatos

```
informacion = os.stat(ARCHIVO)

datos = {
    "nombre": ARCHIVO.name,
    "tamanio": informacion.st_size,
    "lectura": os.access(ARCHIVO, os.R_OK),
    "escritura": os.access(ARCHIVO, os.W_OK)
}
```

Se utiliza os.stat() para consultar propiedades del archivo y os.access() para comprobar los permisos de lectura y escritura.

Respaldo ZIP y metadatos

Respaldo comprimido

La opción Crear respaldo ZIP guarda contactos.dat e indice.json en un solo archivo dentro de respaldos/, después muestra el tamaño original total y el tamaño del ZIP para poder compararlos.

Resultado de una prueba previa	Tamaño
Archivo .dat + índice JSON	1,101 bytes
Respaldo ZIP	588 bytes
Diferencia	513 bytes
Reducción	46.59 %

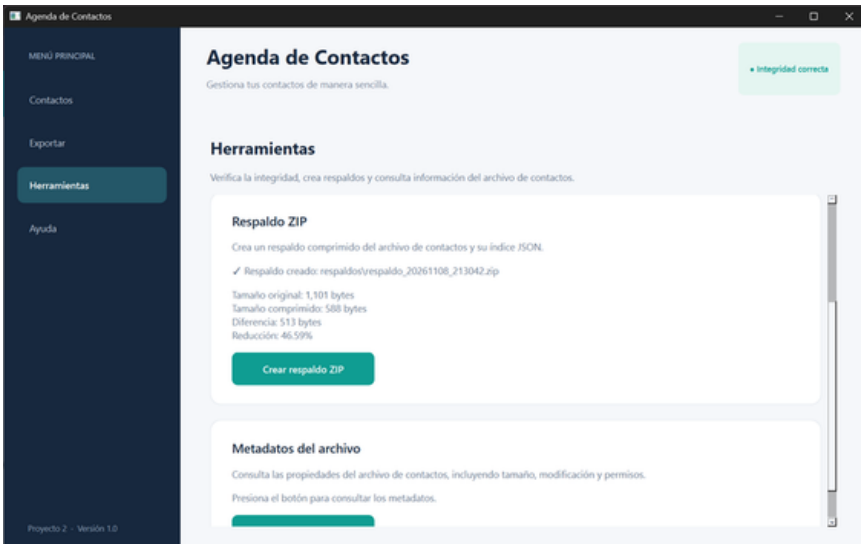
Estas cifras corresponden a una prueba anterior. Si el tamaño cambia al generar el respaldo definitivo, se actualizan con los datos de esa captura.

El ZIP contiene el .dat y su índice; no incluye contactos.sha256. Para verificar una restauración también se necesita conservar el hash correspondiente.

Metadatos y permisos

Mediante os.stat() y os.access(), la aplicación muestra el nombre, la ruta, el tamaño, la fecha de modificación y los permisos de lectura y escritura de contactos.dat.

ZIP y tamaños



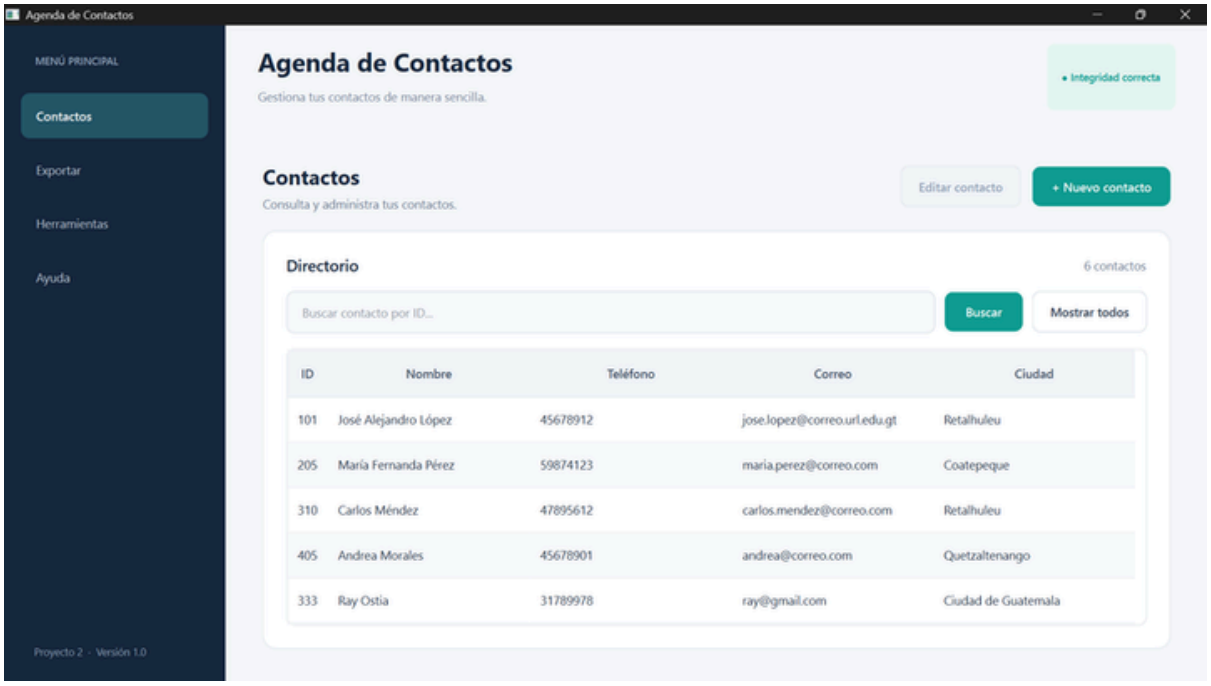
Metadatos



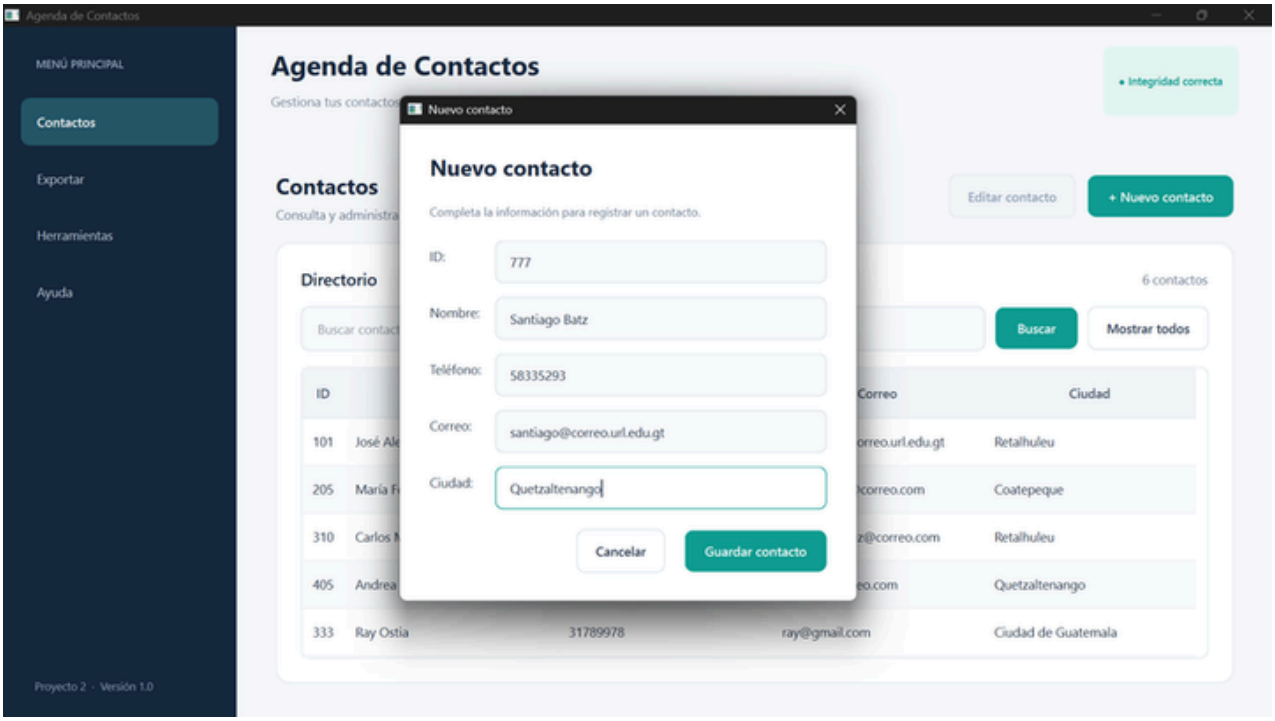
Funcionamiento de la aplicación

Las imágenes de esta sección sirven para mostrar que los botones de la interfaz realizan las operaciones de archivo indicadas

Directorio de contactos

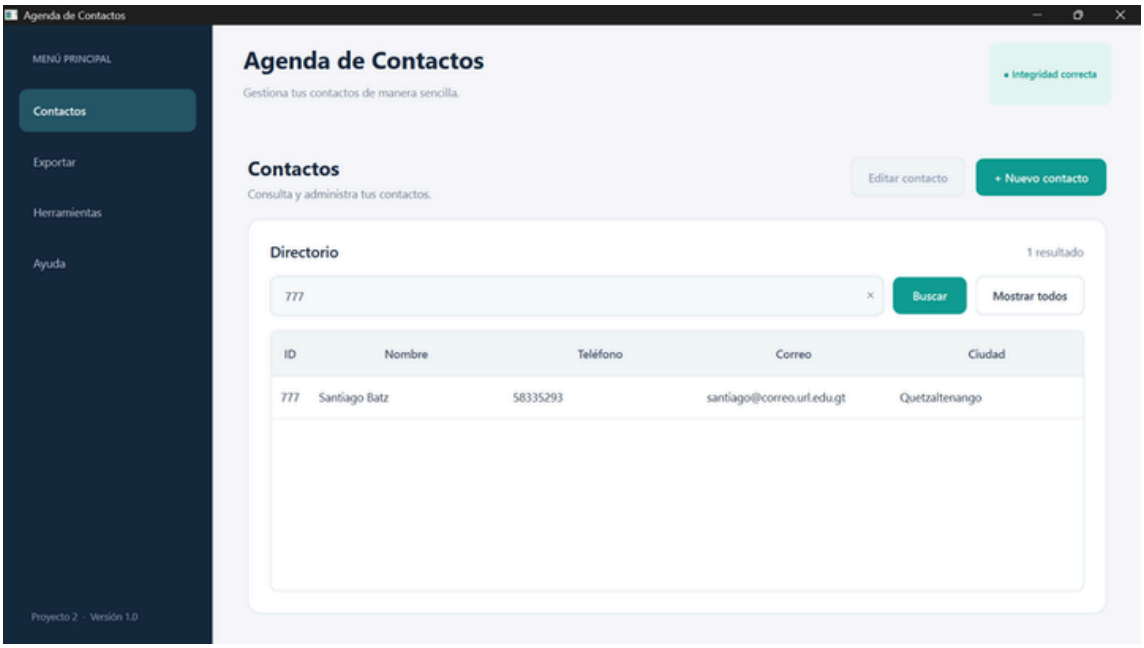


Agregar contacto

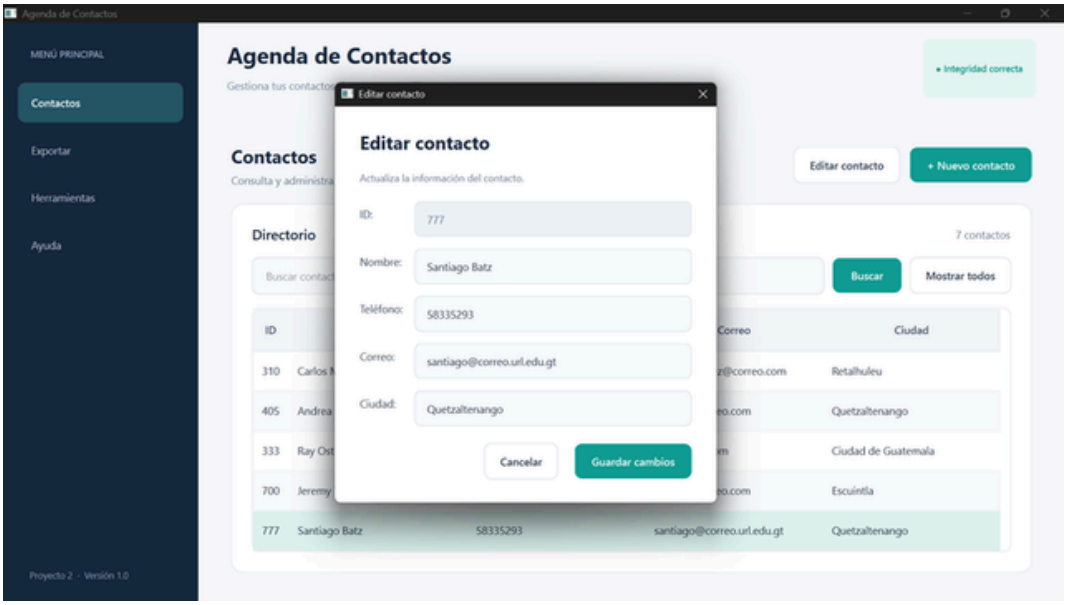


Resultados y conclusión

Búsqueda por ID



Modificar contacto - Antes



Después de modificar datos, se ve así en tabla (tiene ID fijo)



Manejo de errores

La aplicación avisa cuando un ID no existe, cuando se repite un identificador, si un campo no cumple las validaciones o si hay problemas con los archivos y el índice, en esos casos, se muestra un mensaje en vez de continuar como si nada hubiera pasado.

Conclusiones

Con el desarrollo de este proyecto se logró implementar una agenda de contactos que permite guardar, buscar y modificar información mediante registros de tamaño fijo. El uso de un índice JSON y el acceso aleatorio con seek() y tell() permiten localizar los contactos directamente, sin tener que recorrer todo el archivo para cada búsqueda.

También se implementaron funciones para exportar información a JSON y PDF, verificar la integridad de los datos mediante SHA-256, generar respaldos ZIP y consultar las propiedades del archivo. Con esto se pudieron aplicar de forma práctica los principales conceptos de manejo de archivos vistos durante el laboratorio.